

Lab Task: Converting Regular Functions into Arrow Functions

Objective

The objective of this lab is to understand the syntax and usage of **Arrow Functions** in JavaScript by converting traditional function declarations into arrow function notation.

Instructions

1. Read each regular JavaScript function carefully.
 2. Convert each function into its equivalent **arrow function**.
 3. Ensure that your converted function produces the same output as the original function.
 4. Where possible, use the concise arrow function syntax by omitting the `return` keyword and braces `{}` when the function contains only a single return statement.
-

Task 1: Add Two Numbers

```
function add(a, b) {  
  return a + b;  
}
```

Task 2: Subtract Two Numbers

```
function subtract(a, b) {  
  return a - b;  
}
```

Task 3: Multiply Three Numbers

```
function multiply(a, b, c) {  
  return a * b * c;  
}
```

Task 4: Check if a Number is Even

```
function isEven(number) {  
    return number % 2 === 0;  
}
```

Task 5: Find the Square of a Number

```
function square(number) {  
    return number * number;  
}
```

Task 6: Greet a User

```
function greet(name) {  
    return "Hello " + name;  
}
```

Task 7: Find the Maximum of Two Numbers

```
function max(a, b) {  
    if (a > b) {  
        return a;  
    } else {  
        return b;  
    }  
}
```

Task 8: Calculate the Area of a Rectangle

```
function area(length, width) {  
    return length * width;  
}
```

Task 9: Check if a Person is an Adult

```
function isAdult(age) {  
  return age >= 18;  
}
```

Task 10: Convert Celsius to Fahrenheit

```
function celsiusToFahrenheit(celsius) {  
  return (celsius * 9 / 5) + 32;  
}
```

Bonus Challenge

The following functions contain multiple statements. Convert them into arrow functions while preserving their functionality.

Task 11: Calculate Total Price

```
function calculateTotal(price, tax) {  
  let total = price + tax;  
  return total;  
}
```

Task 12: Combine First and Last Name

```
function fullName(firstName, lastName) {  
  let name = firstName + " " + lastName;  
  return name;  
}
```

Task 13: Find the Cube of a Number

```
function cube(number) {  
  let result = number * number * number;  
  return result;  
}
```

Task 14: Calculate the Average of Three Numbers

```
function average(a, b, c) {  
  let sum = a + b + c;  
  return sum / 3;  
}
```

Task 15: Check if a Number is Positive

```
function isPositive(number) {  
  if (number > 0) {  
    return true;  
  }  
  return false;  
}
```

Submission Requirements

- Convert **all 15 functions** into arrow functions.
- Test each function using appropriate sample inputs.
- Use concise arrow function syntax wherever applicable.
- Submit a single JavaScript (`.js`) file containing your solutions.

Total Tasks: 15

Difficulty Level: Beginner to Intermediate

JavaScript Callback Function Programming Problems

Problem 1: Greeting System

Problem Statement

Write a function

```
greetUser(name, callback)
```

The function should:

- Accept a user's name and a callback function.
- Print a welcome message in the format:
 - Hello, <name>! Welcome to our system.
- After printing the greeting, execute the callback function.
- Create a callback function that prints:
 - Goodbye, <name>. Have a nice day!

Example

Input:

Ali

Output:

Hello, Ali! Welcome to our system.

Goodbye, Ali. Have a nice day!

Problem 2: Math Operations

Problem Statement

Create a function

```
calculate(a, b, callback)
```

The function should:

- Accept two numbers and a callback function.

- The callback function should perform a mathematical operation on the two numbers.
- Create separate callback functions for:
 - Addition
 - Subtraction
 - Multiplication
 - Division
 - Modulus
 - Exponentiation

Call the `calculate()` function using each callback and display the result.

Example

Input:

`a = 20`

`b = 5`

Output:

Addition = 25

Subtraction = 15

Multiplication = 100

Division = 4

Modulus = 0

Exponent = 3200000

Problem 3: Array Transformation

Problem Statement

Write a function

```
processArray(arr, callback)
```

The function should:

- Accept an array of numbers and a callback function.
- Apply the callback function to every element of the array.
- Return a new transformed array without modifying the original array.

Create callback functions that:

- Double every number.
- Square every number.
- Cube every number.
- Increment every number by 10.
- Convert every number to its absolute value.

Example

Input:
[2, -3, 5]

Output:

Double:
[4, -6, 10]

Square:
[4, 9, 25]

Cube:
[8, -27, 125]

Increment by 10:
[12, 7, 15]

Absolute Values:
[2, 3, 5]

Problem 4: Task Completion

Problem Statement

Create a function

```
doTask(taskName, callback)
```

The function should:

- Accept a task name and a callback function.
- Print:
 - Doing task: <taskName>
- Simulate completing the task.
- Execute the callback function once the task is complete.

Create callback functions that:

- Print "Task Completed Successfully!"
- Print "Saving task history..."
- Print "Sending completion notification..."

Call `doTask()` using each callback.

Example

Input:
"Download File"

Output:
Doing task: Download File
Task Completed Successfully!

Problem 5: Event Simulation

Problem Statement

Simulate a button click by creating a function

`onClick(callback)`

The function should:

- Simulate a button click event.
- Execute the callback function whenever the button is clicked.

Create callback functions that:

- Print "Button Clicked!"
- Change the button color (display a message).
- Display "Loading...".
- Display "Form Submitted Successfully."

Call the `onClick()` function using each callback.

Example

Output:
Button Clicked!

Problem 6: Authentication Flow

Problem Statement

Write a function

`login(username, password, callback)`

The function should:

- Accept a username, password, and callback function.
- Compare the entered credentials with predefined credentials.
- If the credentials are correct, execute the callback with the message:
Login Successful
- Otherwise execute the callback with:
Login Failed

Use the following credentials:

Username: admin
Password: 12345

Create a callback function that displays the appropriate login message.

Example

Input:
Username = admin
Password = 12345

Output:
Login Successful
Input:
Username = ali
Password = 1111

Output:
Login Failed

Problem 7: Filtering Data

Problem Statement

Write a function

```
filterData(data, callback)
```

The function should:

- Accept an array and a callback function.
- Use the callback function to determine whether each element should remain in the array.
- Return a new filtered array.

Create callback functions to filter:

- Even numbers
- Odd numbers

- Positive numbers
- Negative numbers
- Numbers greater than 50
- Numbers divisible by both 3 and 5

Example

Input:

[12, 25, -8, 45, 60, 73, -15]

Output:

Even Numbers:

[12, -8, 60]

Odd Numbers:

[25, 45, 73, -15]

Positive Numbers:

[12, 25, 45, 60, 73]

Negative Numbers:

[-8, -15]

Greater Than 50:

[60, 73]

Divisible by 3 and 5:

[45, 60, -15]

JavaScript map() Function Programming Problems

Problem 1: Double Every Number

Problem Statement

Write a program that uses the JavaScript `map()` function to create a new array in which every element is doubled.

- The original array should remain unchanged.
- Print both the original and the new array.

Example

Input:
[2, 4, 6, 8]

Output:
Original Array:
[2, 4, 6, 8]

New Array:
[4, 8, 12, 16]

Problem 2: Square Every Number

Problem Statement

Write a program that uses the `map()` function to create a new array containing the square of every element in the given array.

Example

Input:
[3, 5, 7]

Output:
[9, 25, 49]

Problem 3: Add a Constant to Every Element

Problem Statement

Write a program that uses the `map()` function to add **10** to every element of an array and return the transformed array.

Example

Input:
`[12, 8, 25, 30]`

Output:
`[22, 18, 35, 40]`

Problem 4: Convert Celsius to Fahrenheit

Problem Statement

Given an array containing temperatures in Celsius, use the `map()` function to convert each temperature into Fahrenheit.

Use the following formula:

$$F = (C \times 9/5) + 32$$

Return the converted array.

Example

Input:
`[0, 20, 37, 100]`

Output:
`[32, 68, 98.6, 212]`

Problem 5: Replace Negative Numbers with Zero

Problem Statement

Write a program that uses the `map()` function to create a new array in which every negative number is replaced with **0**.

Positive numbers should remain unchanged.

Example

Input:
[-5, 8, -2, 10, -9]

Output:
[0, 8, 0, 10, 0]

Problem 6: Mark Bonus Calculator

Problem Statement

A teacher decides to award **5 bonus marks** to every student.

Given an array of marks, use the `map()` function to create a new array containing the updated marks.

If the updated marks exceed **100**, store **100** instead.

Example

Input:
[75, 98, 63, 100]

Output:
[80, 100, 68, 100]

Problem 7: Even and Odd Transformation

Problem Statement

Write a program that uses the `map()` function to transform an array according to the following rules:

- If the number is even, multiply it by **2**.
- If the number is odd, multiply it by **3**.

Return the transformed array.

Example

Input:
[2, 5, 8, 7]

Output:
[4, 15, 16, 21]

Problem 8: Distance Conversion

Problem Statement

Given an array containing distances in kilometers, use the `map()` function to convert each distance into meters.

Use the following formula:

1 kilometer = 1000 meters

Example

Input:
[2, 5, 8]

Output:
[2000, 5000, 8000]

Problem 9: Electricity Bill Calculator

Problem Statement

An electricity company charges **Rs. 18** per unit of electricity.

Given an array containing electricity units consumed by different customers, use the `map()` function to calculate the bill for each customer.

Formula:

Bill = Units × 18

Return the array of calculated bills.

Example

Input:
[120, 250, 175]

Output:
[2160, 4500, 3150]

Problem 10: Number Transformation Challenge

Problem Statement

Write a program that uses the `map()` function to transform every element according to the following rules:

- If the number is divisible by 5, multiply it by **10**.
- Else if the number is even, multiply it by 2.
- Otherwise, multiply it by 3.

Return the transformed array.

Example

Input:
[5, 8, 7, 10, 13]

Output:
[50, 16, 21, 100, 39]

★ Bonus Challenge 1: Prime Number Transformation

Problem Statement

Write a program that uses the `map()` function to transform an array according to the following rules:

- If the number is **prime**, replace it with its square.
- Otherwise, leave the number unchanged.

Example

Input:
[2, 4, 5, 8, 11]

Output:
[4, 4, 25, 8, 121]

★ Bonus Challenge 2: Custom Number Encoding

Problem Statement

Given an array of integers, use the `map()` function to transform each number using the following formula:

$\text{New Number} = (\text{Number} \times 2) + 5$

After transformation, if the new number is greater than **50**, store **50** instead.

Example

Input:
[5, 10, 15, 30]

Output:
[15, 25, 35, 50]

JavaScript filter() Function Programming Problems

Problem 1: Filter Even Numbers

Problem Statement

Write a program that uses the JavaScript `filter()` function to create a new array containing only the even numbers from a given array.

- The original array should remain unchanged.
- Print both the original and the filtered array.

Example

Input:
[3, 8, 15, 20, 25, 30]

Output:
Original Array:
[3, 8, 15, 20, 25, 30]

Filtered Array:
[8, 20, 30]

Problem 2: Filter Odd Numbers

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the odd numbers from a given array.

Example

Input:
[2, 7, 10, 15, 18, 21]

Output:
[7, 15, 21]

Problem 3: Filter Positive Numbers

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the positive numbers from a given array.

Ignore all negative numbers and zeros.

Example

Input:
[-10, 5, 0, -8, 12, 20]

Output:
[5, 12, 20]

Problem 4: Filter Numbers Greater Than a Given Value

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the numbers greater than 50.

Example

Input:
[25, 60, 48, 90, 15, 75]

Output:
[60, 90, 75]

Problem 5: Filter Multiples of 5

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the numbers that are divisible by 5.

Example

Input:
[12, 15, 20, 23, 30, 41]

Output:
[15, 20, 30]

Problem 6: Filter Prime Numbers

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the prime numbers from a given array.

You must write a separate function to determine whether a number is prime.

Example

Input:
[4, 5, 7, 8, 11, 15, 17]

Output:
[5, 7, 11, 17]

Problem 7: Filter Perfect Squares

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the perfect square numbers.

A perfect square is a number whose square root is an integer.

Example

Input:
[4, 6, 9, 12, 16, 20, 25]

Output:
[4, 9, 16, 25]

Problem 8: Filter Numbers Within a Range

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the numbers between **20 and 50 (inclusive)**.

Example

Input:
[10, 18, 20, 35, 50, 55, 70]

Output:
[20, 35, 50]

Problem 9: Filter Leap Years

Problem Statement

Given an array of years, use the `filter()` function to create a new array containing only the leap years.

A year is a leap year if:

- It is divisible by 400, or
- It is divisible by 4 but not divisible by 100.

Example

Input:
[1999, 2000, 2004, 2005, 2100, 2400]

Output:
[2000, 2004, 2400]

Problem 10: Number Filtering Challenge

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the numbers that satisfy **all** of the following conditions:

- The number is greater than **20**.
- The number is less than **100**.
- The number is even.

- The number is **not** divisible by 3.

Example

Input:
[18, 24, 28, 36, 44, 75, 90, 98, 102]

Output:
[28, 44, 98]

★ Bonus Challenge 1: Filter Armstrong Numbers

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the **Armstrong numbers**.

For this problem, assume the array contains only **three-digit numbers**.

Example

Input:
[153, 125, 370, 371, 407, 500]

Output:
[153, 370, 371, 407]

★ Bonus Challenge 2: Filter Numbers Divisible by Multiple Values

Problem Statement

Write a program that uses the `filter()` function to create a new array containing only the numbers that are divisible by **2, 3, and 5** simultaneously.

Example

Input:
[10, 15, 30, 45, 60, 75, 90, 100]

Output:
[30, 60, 90]
